INDEPENDENT REPOSITORY REVIEW

Asymptotic 1.8.0

Correctness, precision contracts, performance, and Wolfram Language integration

Prepared for Vladimir Reshetnikov

9 September 2026

Reviewed repository VladimirReshetnikov/Asymptotic

Pinned commit `07a9781212beb2eeb9ff16aa625b50ac27974078`

Native audit kernel Wolfram Language 15.0.0, Linux x86-64

Distribution verified 573,940-byte standalone source

Principal result. This is a substantial, useful extension of Wolfram Language, not a replacement for its asymptotic system. Its strongest contributions are exact real-exponent inversion, explicit remainder transport, retained inverse cores, and branch-aware result objects. Four concrete defects or resource-control gaps were reproduced in a native kernel: lost ambient assumptions, a fractional-power branch-guard bypass, eager dense allocation from sparse rational exponents, and a generic expansion loop that exceeds `MaxTerms`.

This report distinguishes native observations, source-level deductions, upstream validation claims, and proposed changes. It does not claim a full-suite run, exhaustive verification of every module, or general performance superiority. The accompanying archive contains bounded reproducers, desired-contract tests, an experimental patch generator, and machine-readable evidence.

Contents

1 Executive assessment

1.1 Recommendation

Continue developing the package, but make *assumption provenance and resource safety* the next stabilization milestone before adding another broad family of special-function adapters. The implementation has moved well beyond a prototype: its ordinary inverse engine, weighted support enumeration, retained-core expansions, composite error envelopes, and exact interval certificate machinery are meaningful engineering work. Nevertheless, the audit demonstrates that a result can be simplified under a condition not recorded in its metadata, and that one public route can admit a fractional-power observable without the real-branch evidence required by another route. These are central contract issues, not cosmetic discrepancies. The native observations and source traces appear in Section 6.

The correct positioning is a *specialized generalized-series and inverse-expansion layer on top of Wolfram Language*. Native `Series`, `InverseSeries`, `ComposeSeries`, `Asymptotic`, and `AsymptoticSolve` already cover important parts of the problem. The repository itself calls native `Series`, symbolic simplification, equation solving, and special-function facilities. Its added value is the explicit representation, specialized inversion algorithms, reusable precision information, and carefully selected extensions—not the invention of asymptotic computation in the host language.[19, 21, 22, 23, 24, 12, 3]

1.2 Prioritized findings

ID	Priority	Finding	Evidence
F01	High	Ambient assumptions can affect coefficients while the object records `Assumptions -> True`.	Native reproduction and source trace.
F02	High	Wrapped `SeriesObservable` fractional powers bypass the pure-remainder branch guard.	Native reproduction and source trace.
F03	High	Optional `SeriesData` conversion eagerly allocates a dense rational grid, bypassing sparse resource limits.	Native memory measurements; exact allocation formula.
F04	High	Generic unit-series expansion can return more blocks than `MaxTerms`, with no matching depth bound.	Native reproduction and source trace.
F05	Medium	A coefficient recurrence that has become identically zero continues generating powers unnecessarily.	Source-level proof.
F06	Medium	Resource budgets have inconsistent units; simplification, factorization, and total work are not uniformly bounded.	Source/design assessment.
F07	Medium	Selected native tests and builder-only CI are insufficient release gates for cross-cutting changes.	Explicit upstream validation records.
F08	Medium	The result-head rename breaks saved expressions and patterns; migration remains a user task.	Documented behavior.
F09	Low	The root license is MIT No Attribution, but paclet metadata says MIT.	Source metadata comparison.
F10	Low	`InverseCertificate` defaults its interval to `Automatic`, then rejects that value.	Source-level API observation.

These entries are not ten independently demonstrated mathematical wrong answers. F01 and F02 are correctness/contract defects; F03 and F04 are reproduced resource-control gaps; F05 is a proved wasted-work path; the remaining entries are engineering, compatibility, or usability findings. The table deliberately does not count documented restrictions as undiscovered bugs.

1.3 What is already done well

The current version already preserves complete logarithmic blocks, uses exact exponent comparison rather than floating-point sorting, checks Newton residuals symbolically, and distinguishes the explicit forward source from a finite model. Differentiating an arbitrary $\mathcal{O}$-remainder is not silently assumed valid: derivative contracts are part of the calculus. Composite arithmetic retains absolute error envelopes instead of manufacturing a fictitious single scale. A numerical check is not presented as an interval proof. Exact termination is checked against the original expression rather than inferred from a finite truncation. These safeguards should be retained while repairing the inconsistencies found here.[2, 5, 7, 8, 9, 13]

2 Snapshot, methodology, and limits of the evidence

2.1 Exactly what was reviewed

The review is pinned to commit `07a9781212beb2eeb9ff16aa625b50ac27974078`, whose commit timestamp is 9 September 2026 at 19:01:30 UTC and whose subject is "Rename the result head to GeneralizedSeries and simplify series operations." The paclet declares version 1.8.0 and Wolfram Language 15.0 or later. Its canonical implementation is modular; the repository-root `AsymptoticInverse.wl` is a generated standalone distribution. The development and validation records describe 38 included kernel sources, preserved module order, and normalized-source hashes.[1, 16, 15, 14]

The successful native audit loaded the pinned standalone with `URLDownload` followed by ordinary `Get`. Its byte count was 573,940 and its SHA-256 was

```
b2aebac8176649ff53634816f17e92c8f0ccb397f7606a5ac819b6f157b6d7ed
```

The kernel reported `15.0.0 for Linux x86 (64-bit) (May 6, 2026)`. This is distinct from the repository's reported 15.0.1 Windows validation environment. Both are within the declared 15.0+ requirement; the observations must not be silently relabeled as 15.0.1 results.

2.2 Evidence classes

Native observations are outputs actually returned by the connected Wolfram evaluator. Four findings, the dense-grid measurements, polynomial reversion, and the irrational inverse comparison belong to this class. **Source deductions** follow from inspected definitions and mathematical reasoning, including the exact dense-grid size and the zero-recurrence early-exit argument. **Upstream evidence** consists of the repository's own test and benchmark records; those counts are not new runs by this audit. **Proposals** are changes or future designs and are not presented as already implemented.

The evaluator was intermittent: initial requests and several later batches failed with service or transport errors. Other calls succeeded. A failed tool transport is not evidence that a mathematical algorithm failed or timed out. The useful successful results are transcribed in `evidence/native_verified.json`; the runnable probes are in `code/reproduce.wl` and `code/comparison.wl`.

2.3 Coverage and exclusions

Source inspection concentrated on the main kernel, coefficient normalization, sparse jets, forward and inverse object construction, series operations, ordinary arithmetic dispatch, composite

envelopes, Fourier coefficients, incremental inversion, exact termination, interval certificates, the native special-function adapter, Gamma/Barnes inverse construction, package metadata, current user documentation, and release/build records. Appendix A gives the relevant files and inspected regions.

This was not a line-by-line verification of every one of the 38 modules or every historical report. The full upstream Wolfram suite was not executed here, nor were the upstream Python builder tests rerun. The 18 local Python tests in this archive check independent arithmetic facts and the mechanics of the proposed source transformations on explicit anchor fixtures. They do *not* emulate Wolfram Language or establish correctness of the package. This distinction is important because a large numerical test count can otherwise give a false impression of coverage.

3 Architecture and implemented capabilities

3.1 The ordinary representation

An ordinary expansion is expressed in a positive local coordinate $w \to 0^+$, using blocks

$$w^\beta P(\log w), \qquad \beta \in \mathbb{R},$$

where P is a polynomial and admissible exponents are represented exactly. The coordinate is $x - x_0$, $x_0 - x$, $1/x$, or $-1/x$, depending on the endpoint and side. Coefficients at the same weight are combined before zero blocks are removed and term counts are determined. This makes the object materially different from a list of terms sorted by numerical exponent approximations.[2, 3]

At the lower level, a precision-tracked jet has the form

$$J = (T, P, D), \qquad F(w) = \sum_{(\beta,Q)\in T} w^\beta Q(\log w) + \mathcal{O}\Big(w^P (1 + |\log w|)^D\Big).$$

`GeneralizedSeries[association]` is the public result wrapper. It can store a prefactor, offset, source and target conditions, a frontier term, replay/refinement data, and scale-specific coefficients. `Normal[s]` deliberately removes this information. The wrapper's native-looking display is an interpretation of an object, not a native `SeriesData` value. Read-only interpretation boxes are a sensible defense against editing a displayed coefficient while accidentally preserving stale metadata.[2, 5, 3]

3.2 Ordinary inversion

For an admitted source model

$$f(u) = y_0 + a u^p \left(1 + \sum_{j=1}^{m} u^{\delta_j} P_j(\log u)\right), \qquad \delta_j > 0,$$

the inverse is expanded around an appropriate positive target uniformizer. The ordinary algorithms include direct Lagrange coefficients, grouped Lagrange coefficients, and Newton refinement in weighted jets. Incremental enumeration advances whole equal-weight layers, which is essential when distinct multi-indices produce the same exponent. Reusable state is returned by value rather than placed in an unbounded global coefficient cache.[2, 8]

The incremental implementation already caches powers of nonconstant logarithmic coefficient polynomials, accounts for cache entries against available index-budget slots, and evicts cache entries rather than making cache saturation an input failure. Newton refinement doubles a precision target and verifies that the normalized residual vanishes below that target. The grouped method combines terms sharing an Euler operator before differentiating them. These are substantive optimizations, not merely different public method names.[8]

3.3 Scale extensions

The package is not limited to the ordinary power–log class. Its documented constructors cover retained exact inverse cores, logarithmic coordinates and selected reciprocal-logarithmic hierarchies, finite flat exponential sectors, finite Fourier coefficients in logarithmic phase, and selected special-function inverse families. These are separate admitted calculi, not one completely general transseries field. Their orders, remainders, and available operations differ.[3, 10]

Gamma and Barnes inverse expansions are particularly useful examples. The Gamma inverse retains the Lambert core

$$X = \frac{L}{W(L/e)},$$

where L is the normalized logarithmic target. Its correction coefficients are complete polynomials in $1/\log X$ at successive powers of $1/X$. For the Barnes construction the corresponding core is

$$X = \sqrt{\frac{4L}{W(4L/e^3)}},$$

with a shifted coefficient coordinate $1/(\log X - 1)$. The implementation records these as finite Poincaré constructions rather than convergent inverses of an infinite Stirling series. Its recognizer admits particular affine source/target transformations and powers, not arbitrary compositions of Gamma and Barnes functions.[11]

The native special-function forward adapter reads structured output from Wolfram `Series`, identifies carriers and amplitude rows, and transports absolute errors. It does not indiscriminately accept a finite expression with no remainder as exact: the inspected path requires exact source equality in that situation. Nonexact exits require the native error to be asymptotically smaller than the chosen explicit truncation envelope. This is an integration layer over the host system, not independent rederivation of every special-function expansion.[12]

3.4 Arithmetic and validation layers

Ordinary `Plus`, `Times`, and supported function applications have guarded dispatch. The explicit `SeriesNormalize` entry holds the expression tree so that reciprocals can be checked before ordinary evaluation cancels denominators. Unsupported combinations can become composite results with separate absolute error scales; a composite result then refuses a fictitious single exponent cutoff. The distinction between a failed proof obligation and an unsupported representation is therefore part of the API.[6, 7]

The verification layer has three different purposes. `InverseResidual` can check formal composition with the finite model. Numerical checks compare approximations at chosen targets but do not prove bounds. `InverseCertificate` uses exact interval enclosures on a supplied finite interval to establish a unique real root of the stored explicit function. These distinctions are documented and should not be collapsed into one boolean “verified” label.[2, 13, 3]

4 Comparison with built-in Wolfram Language

4.1 A capability comparison, not a replacement claim

Task	Native facilities	Package contribution or boundary
Taylor, Laurent, and Puiseux work	`Series` and arithmetic on `SeriesData`.	A unified explicit envelope and exact weighted blocks; native tools remain the natural baseline.
Logarithmic terms	`Series` already handles certain logarithmic expansions.	Complete polynomial-log blocks and explicit remainder log degree, including inversion workflows.
Series reversion	`InverseSeries` reverts suitably structured native series.	Real-exponent weighted inversion and selected nonordinary scales.
Implicit asymptotics	`AsymptoticSolve` handles local solution branches, systems, real/complex settings, and inferred scales.	A specialized source-endpoint inverse API, retained model, and follow-on precision operations.
General asymptotics	`Asymptotic` includes expressions, integrals, transforms, and other built-in workflows.	Narrower overall scope; more explicit object-level contracts for the admitted classes.
Composition	`ComposeSeries` and substitution of suitable native series.	Generalized-jet error transport plus selected scale-specific composition operations.
Special functions	Broad built-in symbolic and asymptotic infrastructure.	Real-domain admission, normalization, carrier decomposition, and retained-core inverse adapters.
Exact irrational weights	Native output may factor nonstandard powers outside `SeriesData`.	Explicit sparse support at exact real weights, without requiring one rational exponent grid.
Differentiation	Native series differentiation and symbolic `D`.	Explicit remainder-derivative assumptions for nonanalytic asymptotic objects.
Integration	Native series integration is available.	No general public generalized-series integration operation appears in the current overview.
Remainder meaning	Native series order and asymptotic relations.	Explicit power/log envelopes, scale metadata, and selected finite-interval certificates.
Complex and multivariate work	Native facilities include complex centers and systems/multivariate expansions.	Current focus is selected real, essentially one-variable approaches; not a universal substitute.

The native capabilities in this table are described in the official documentation; the package column is based on its current guide and inspected implementation.[19, 20, 21, 22, 23, 24, 3, 12]

4.2 Native experiment: polynomial reversion agrees

For the positive local inverse of $x + x^2$, the audit evaluated the package at exclusive cutoff 6 and compared it with

```
InverseSeries[x + x^2 + O[x]^6, y]
```

The package returned

$$y - y^2 + 2y^3 - 5y^4 + 14y^5 + \mathcal{O}(y^6),$$

and the native result was

```
SeriesData[y, 0, {1, -1, 2, -5, 14}, 1, 6, 1]
```

Thus the finite coefficients and power order agree. For this case, the reason to use the package is its subsequent metadata/refinement workflow, not an unavailable native inverse operation. The experiment is recorded in `native_verified.json`; it is not a throughput benchmark.

4.3 Native experiment: an irrational inverse is a genuine useful case

The successful package call was

```
AsymptoticInverse[x + x^Sqrt[2], {x, 0}, y,
  SeriesTermGoal -> 3]
```

and returned

$$g(y) = y - y^{\sqrt{2}} + \sqrt{2}\, y^{2\sqrt{2}-1} + \mathcal{O}\left(y^{3\sqrt{2}-2}\right), \qquad y \to 0^+. \tag{1}$$

Its formal residual check reported `ZeroBelowCutoff -> True`, an empty residual support, and normalized residual zero, explicitly scoped to the finite forward model.

On the same reported kernel version, these two native requests did not produce the corresponding finite expansion:

```
AsymptoticSolve[x + x^Sqrt[2] == y, x -> 0, y -> 0,
  Reals, SeriesTermGoal -> 3, Direction -> "FromAbove",
  GenerateConditions -> True]

Asymptotic[InverseFunction[Function[t, t + t^Sqrt[2]]][y],
  y -> 0, SeriesTermGoal -> 3, Assumptions -> y > 0,
  GenerateConditions -> True]
```

The first remained an unevaluated `AsymptoticSolve`; the second returned the inverse-operator application and emitted messages. An earlier trial incorrectly put a variable-dependent assumption in `AsymptoticSolve`; the kernel warned that it was ignored. The corrected call above removes that option, so the comparison does not depend on the warned-about input.

These observations establish usefulness for these specific calls on 15.0.0, not a theorem that Wolfram Language cannot solve the problem by any transformation. Indeed, marker expansions, coefficient matching, symbolic differentiation, and manual change of variables are ways to build such algorithms in the language. The package makes that specialized workflow systematic and reusable.

4.4 Term goals are not interchangeable

The native audit also evaluated `Asymptotic[Erfc[x], x -> Infinity, SeriesTermGoal -> 3]` and obtained two nonzero amplitude terms,

$$\frac{e^{-x^2}}{\sqrt{\pi}}\left(\frac{1}{x}-\frac{1}{2x^3}\right).$$

With native term goal 5 the result additionally contained $3/(4x^5)$. The package documents complete nonzero block counting, and, for multiple carriers, counting separately within each carrier. Therefore equal numerical `SeriesTermGoal` options are not by themselves a fair matched-accuracy comparison. This audit does not call the native goal behavior a bug; it records the observed output and recommends normalizing retained scales before comparing systems.[3]

4.5 Interop rules worth enforcing

Do not translate a package cutoff h mechanically into `Series[...,n]`. Ordinary package cutoffs are exclusive, while native polynomial order notation retains terms through the requested degree. Moreover, some package cutoffs apply inside an exact prefactor or in a core coordinate. An integer such as 5 can therefore describe three different notions of order in superficially similar calls.[19, 3]

The package guide already warns that conversion to native `SeriesData` may hide the logarithmic degree of an error. Preserve the original remainder descriptor whenever error transport matters. Conversely, importing native output should retain its coordinate, branch conditions, carrier structure, and nonexactness rather than merely reading `Normal`. In particular, a native `Asymptotic` approximation is not automatically a pointwise numerical certificate.[3, 20, 23]

5 Mathematical contracts that the implementation must preserve

5.1 Ordering, local finiteness, and complete blocks

For fixed real $a > b$ and fixed log degrees, powers dominate logarithms:

$$w^{a-b}(1+|\log w|)^d \longrightarrow 0 \quad (w \to 0^+).$$

Consequently, one may order *finite* power–log blocks first by their power weight. With finitely many positive gaps δ_j, the index set

$$\{k \in \mathbb{N}^m : k \cdot \delta < C\}$$

is finite, because each $k_j < C/\delta_j$. Irrational gaps do not invalidate finite-cutoff enumeration. They do, however, make a dense rational-grid representation inappropriate. These elementary facts explain both the correctness of weighted truncation and the resource risk when a gap is tiny.

At a fixed weight, the entire polynomial in $L = \log w$ must be retained or moved to the remainder. Counting individual summands in that polynomial as independently ordered blocks can lose cancellation and misstate the error. The current merge-and-trim design gets this central point right.[2]

5.2 Error transport is an algebra, not a display convention

Suppose $F = F_0 + \mathcal{O}(R_F)$ and $G = G_0 + \mathcal{O}(R_G)$, where the displayed envelopes are nonnegative. Then

$$FG = F_0G_0 + \mathcal{O}(|F_0|R_G + |G_0|R_F + R_FR_G)\,.$$

The cross term cannot be dropped universally. For ordinary jets the leading weights and logarithmic degrees compress this envelope into a power–log precision pair; at a truncation boundary, omitted products can raise the required logarithmic degree. The inspected `pMul` explicitly accounts for a boundary-product degree, which is an important correctness feature.[2, 7]

Reciprocals and fractional powers require more than a magnitude envelope. A reciprocal needs eventual separation from zero. A real noninteger power needs a proved real branch, usually eventual positivity. An $\mathcal{O}(w^4)$ class contains both w^4 and $-w^4$. This is the mathematical reason the public guard in F02 must live at a shared semantic layer, not only in a convenient entry function.

For exponentials, if $F = A + \varepsilon$ with $\varepsilon = o(1)$, then

$$e^F = e^A(1 + \mathcal{O}(\varepsilon)).$$

A large exact A can be retained as a carrier. But a nonvanishing absolute uncertainty in F cannot simply be called a small relative uncertainty in e^F. The package's explicit restriction on exponentiating an insufficiently precise argument is therefore justified, not an arbitrary unsupported case.[5]

5.3 Why differentiation needs a separate contract

The implication $r(w) = \mathcal{O}(w^P) \Rightarrow r'(w) = \mathcal{O}(w^{P-1})$ is false without regularity information. For example,

$$r(w) = w^P \sin(w^{-2})$$

has the stated magnitude bound but a derivative containing $-2w^{P-3}\cos(w^{-2})$. A general asymptotic object must track a differentiability/remainder hypothesis separately. The repository's `RemainderDerivativeOrder` mechanism is a positive design decision. It should be carried through every representation conversion and operation rather than inferred from a smooth-looking finite expression.[5, 3, 27]

5.4 The Euler coefficient formula and its useful consequences

For the finite model from Section 3, put $n = |k|$, $\omega = k \cdot \delta$, and $B_k(L) = \prod_j P_j(L)^{k_j}$. The inspected ordinary inverse coefficient implementation applies, for $n \geq 1$,

$$C_k(L) = \frac{(-1)^n r}{p^n \prod_j k_j!} \prod_{j=1}^{n-1} (\mathcal{D}_L + r + \omega + pj)\, B_k(L). \tag{2}$$

The zeroth coefficient is one. This yields the unit correction in the expansion of the chosen inverse observable. Grouping equal (n, ω) before applying the differential operator is valid because the operator is linear and identical for all such terms.[8, 4]

As a simple check, set $p = r = 1$, one gap $\delta = 1$, and $P(L) = 1 + L$. The first coefficient is $-(1 + L)$; the second is

$$\frac{1}{2}(\mathcal{D}_L + 4)(1 + L)^2 = 3 + 5L + 2L^2,$$

which gives the documented inverse of $x + x^2(1 + \log x)$. With one constant coefficient and gap $\sqrt{2} - 1$, the first coefficients instead give equation (1). These independent algebraic checks help separate a sound coefficient formula from bugs in wrappers or resource handling.

5.5 A residual is not automatically an inverse error bound

If g is a candidate inverse and $\rho = f(g) - y$, a small ρ is useful only relative to the slope of f. On an interval containing the relevant root, a lower bound $|f'| \geq m > 0$ yields

$$|g - f^{-1}(y)| \leq |\rho|/m.$$

For a model with $f(u) \sim au^p$, this suggests the shift from a forward error of order u^q to an inverse displacement error of order u^{q-p+1}, subject to the corresponding local derivative and branch hypotheses. One must not label a formal residual identity for a truncated model as a bound for an unknown source remainder. The certificate and model-scope labels inspected in this repository explicitly recognize that distinction.[2, 13]

6 Reproduced defects and source-level findings

6.1 F01: Ambient assumptions are used but not retained

Priority: High.

Evidence: native reproduction; current coefficient-normalization source.

```
s = Assuming[a > 0,
  AsymptoticExpansion[Sqrt[a^2] + x, {x, 0, 2}]];
{Normal[s], s["Assumptions"], s["TargetDomain"]}
(* {a + x, True, x > 0} *)
```

Under the actual condition $a > 0$, replacing $\sqrt{a^2}$ by a is correct. The defect is that the resulting object records no such condition. Its expression at $a = -1$ is $-1 + x$, whereas the original source at that parameter value is $1 + x$. The asymptotic error is exactly zero in this finite example, so an omitted higher-order term cannot absorb the discrepancy. The native response confirmed the expression, stored assumption, target domain, and negative-parameter substitution.

The inspected `coefCanon` eventually calls `Simplify[e, ass]`; `polyCanon` and several sign/equality checks use the same second-argument form. The public option default is `Assumptions -> True`, and object construction stores the explicit `ass` value. Ambient `$Assumptions`, introduced by `Assuming`, can therefore influence a simplification without being included in that stored value. The observed example makes this more than a hypothetical concern about global state.[2, 3, 25, 26]

Consequences. A later operation can trust an overly broad domain, a serialized object can outlive the assumptions under which it was built, and a refinement cache can reuse coefficients under a different effective environment. The current example proves coefficient specialization with lost metadata; it does not establish that every constructor or cache is affected identically. Those entry paths need targeted testing.

Recommended repair. Introduce one evaluation-context boundary. At public entry, determine the effective assumptions according to a documented option policy, capture them as a value, and store them. During the mathematical computation, isolate the ambient variable with `Block[{$Assumptions = True}, ...]` and pass only the captured assumptions to proof and simplification operations. A standard-compatible default can be delayed evaluation of `$Assumptions`; an explicitly supplied option should have a defined overriding or combining policy. Existing-object arithmetic should rely on the operands' recorded conditions, not silently specialize under a new ambient context.

Do not repair only `coefCanon`: branch selection, equality tests, native `Series` calls, model recognition, and refinement all depend on the same semantic environment. Cache keys must

include the effective context and any coordinate/branch selection that changes a coefficient. If current defaults are intentionally kept, the implementation must at least prevent unrecorded ambient assumptions from influencing results.

Immediate workflow safeguard. Evaluate with ambient assumptions cleared and provide the intended conditions explicitly:

```
Block[{$Assumptions = True},
  AsymptoticExpansion[Sqrt[a^2] + x, {x, 0, 2},
    Assumptions -> a > 0]]
```

This is a scope-control pattern, not a substitute for repairing the library. It also avoids presenting a broad global-state patch as tested when only one constructor has been checked. The supplied hardening patch does not claim to fix F01.

Acceptance tests. Cover positive, negative, unknown, and contradictory parameter conditions; repeat with `Assuming`, explicit options, saved objects, refinement, arithmetic performed under a new ambient context, and source-coordinate transformations. A result must either retain every condition used or remain valid without the unrecorded specialization. Explicitly test a symbol whose sign is unknown: a conservative unsimplified coefficient is preferable to an unjustified sign choice.

6.2 F02: A wrapped observable bypasses the real-power guard

Priority: High.

Evidence: native reproduction; shared low-level power evaluator.

```
s = AsymptoticExpansion[-x^4, {x, 0, 1}];
{Normal[s], s["Remainder"]}
(* {0, PowerLogRemainder[x, 4, 0]} *)

SeriesPower[s, 1/2]
(* Failure["UnknownLeadingTerm", ...] *)

SeriesObservable[s, 1 + Sqrt[z], z]
(* GeneralizedSeries with finite expression 1
   and remainder PowerLogRemainder[x, 2, 0] *)
```

For positive real x, the actual observable is

$$1 + \sqrt{-x^4} = 1 + ix^2.$$

It is not real. The returned error magnitude happens to bound this complex difference, so the precise allegation is a *real-branch/admission contract violation*, not that $\mathcal{O}(x^2)$ is an invalid complex magnitude bound.

The direct `seriesPower` path rejects a noninteger power of an empty retained jet with finite remainder. However, generic `seriesJetApply` sends a constant exponent to `fwdPower`. In that lower-level function, the empty-jet branch accepts any positive exponent and scales the remainder power and log degree. It executes before a positive leading coefficient can be established. Wrapping `Sqrt[z]` in `1 + ...` avoids the special top-level shortcut to `seriesPower`.[2, 5]

Repair included in the experimental patch. Put the uncertain-real-power guard in `fwdPower`, after the nonnegative integer case but before scaling an empty finite-remainder jet. Preserve the exact-zero case separately. A failure of type `UnknownLeadingTerm` permits a constructor that supports adaptive refinement to discover a leading block, rather than inventing a branch. Richer sign provenance can later admit safe cases; the magnitude envelope alone is

insufficient.

The focused native patched check returned `Failure["UnknownLeadingTerm", ...]` for the wrapped observable. This validates the specific correction path, not every fractional power in every scale. Regression coverage should compare `SeriesPower`, ordinary powers, `SeriesNormalize`, direct `SeriesObservable[s,Sqrt[z],z]`, and wrapped algebraically equivalent observables. Include exact zero, a negative known leading coefficient, an unknown-sign leading coefficient, a positive leading coefficient, pure remainder inputs of either source sign, and rational exponents with both signs.

6.3 F03: Sparse rational exponents cause eager dense allocation

Priority: High.

Evidence: native peak-memory observations; exact source allocation size.

The ordinary forward and inverse object builders attempt a native `SeriesData` cache. Their conversion helpers form a common rational denominator, determine integer grid endpoints, and allocate

```
coeffs = Table[0, {nmax - nmin}];
```

before inserting the sparse coefficients. There is no dense-span budget in these helpers. The forward cache is built eagerly even when the user only needs the sparse result or `Normal`.[2]

Consider the public input

```
AsymptoticExpansion[x + x^(1 + 1/N) + x^2,
  {x, 0, 3/2}, "MaxTerms" -> 10]
```

with an even positive integer $N > 2$. The retained weights are 1 and $1 + 1/N$; the omitted source term gives remainder order 2. The rational grid has denominator N, lower index N, and upper index $2N$. Hence the cache allocates exactly N entries although there are only two retained blocks. With $N = 10^9$, the source requests a billion-entry intermediate list. That large case was *not* executed.

N	Retained blocks	Additional peak bytes	Elapsed seconds
1,000	2	89,576	0.007882
100,000	2	1,655,616	0.037655
1,000,000	2	16,055,384	0.294606

Table 3: Native audit observations for F03. These are single bounded runs on the remote 15.0.0 Linux kernel, not a controlled performance comparison. Additional peak memory was measured with `MaxMemoryUsed[expression]`.

Native `SeriesData` may subsequently compress or trim parts of the coefficient sequence. That does not remove the eager intermediate allocation. Nor is the issue an inherent limitation of the sparse series object: it is an interoperability cache imposing a much larger representation.

Preferred repair. Make native conversion lazy, explicit, and budgeted. Compute denominator, span, and density before allocation. If conversion is too large, return a `Missing` descriptor for the optional property while preserving the valid sparse object. A missing compatibility cache must not become a failure to compute the sparse expansion. Apply this policy to both forward and inverse helpers, and include integer bit length and least-common-multiple growth in resource accounting.

Minimal patch included. The accompanying candidate checks $n_{\max} - n_{\min} > 100000$ and returns `Missing["DenseRepresentationLimit", ...]` before allocating. The fixed threshold is a narrow safety cap, not a complete global budget policy. In the focused native check, the million-grid example retained $x + x^{1000001/1000000}$, while its native cache reported the required and allowed sizes. The long-term interface should expose a named dense-conversion budget and avoid building the cache at all until requested.

6.4 F04: Generic unit-series loops evade the advertised budget

Priority: High.

Evidence: native reproduction; per-operation versus accumulated work.

```
s = AsymptoticExpansion[Exp[x^(1/50)], {x, 0, 1},
  "MaxTerms" -> 10];
s["ReturnedTermCount"]
(* 50 *)
```

The kernel returned 50 complete blocks and `PowerLogRemainder[x,1,0]`. The guide calls `MaxTerms` a resource budget for exact expansion operations; this path does not enforce a corresponding output or iteration bound.[3, 2]

For $U = w^{1/N}$, each successive power is still a single monomial. A product routine that limits the number of retained multiplication pairs therefore sees only one pair at each step. But

$$e^U = \sum_{k \geq 0} \frac{w^{k/N}}{k!}$$

contains N nonzero blocks below exclusive cutoff 1. In general the number of positive powers k with $k\delta < H$ is

$$\max\{0, \lceil H/\delta \rceil - 1\}.$$

The `jetPowerSeries` loop has no matching bound on k or on the accumulated support. Repeated `jetAdd`/`jetMerge` work can consequently grow with that support while each individual multiplication remains within its local pair limit.

Repair included. Add a depth guard and an accumulated-support guard to the generic loop, with correct ordering around exact termination and the cutoff-empty test. Apply an analogous guard to unit block composition. The native patched probe returned `Failure["ResourceLimit", ...]` instead of a 50-block result under budget 10. Other algorithms already enforce some depth or frontier budgets, so this repair closes a gap rather than inventing resource checks from scratch.[8]

A structural guard should not be confused with a wall-clock guarantee. One retained polynomial can itself have a huge coefficient expression. A future budget object should distinguish output support, generated products, iteration depth, coefficient complexity, integer size, and total work. Raising a budget should normally reveal more of the same consistent result, not change a branch decision or silently truncate an unrecorded error.

6.5 F05: An annihilated composition recurrence does not stop

Priority: Medium.

Evidence: source-level proof; focused patched native control.

The block-composition routine expands

$$(1+U)^a P(L+\log(1+U)) = \sum_{k\geq 0} Q_k(L)U^k$$

using

$$Q_0 = P, \qquad Q_{k+1} = \frac{(a-k)Q_k + Q_k'}{k+1}.$$

The current loop computes a new power of U, then executes `Continue[]` when Q_k is proved zero. For $a = 0$, $P = 1$, all coefficients after Q_0 vanish, yet a tiny positive valuation can make the routine continue until the weight cutoff is reached.[2]

Proposition 6.1. *If the coefficient polynomial Q_k in this recurrence is identically zero under the fixed valid coefficient assumptions, then every later Q_j, $j > k$, is zero under those assumptions.*

Proof. The derivative of the zero polynomial is zero, so the displayed linear recurrence maps zero to zero. Induction completes the argument. □

Thus the safe optimization is to test the newly computed coefficient before multiplying by another U, and `Break[]` rather than continue when it vanishes identically. This is not the unsafe rule "stop after one zero term" for an arbitrary coefficient generator: it relies on this specific homogeneous recurrence. The included patch uses that argument. A native patched call with $U = w^{1/1000000}$, $a = 0$, and $P = 1$ returned the constant jet $\{\{0, 1\}\}$; the zero-coefficient exit avoids the million-step recurrence.

7 Performance, resource architecture, and numerical workflow

7.1 F06: budgets need consistent units and a whole-operation envelope

The code has several useful local controls: product-count limits, index-frontier limits, polynomial leaf-count checks, a finite Taylor-order ceiling in one generic path, fixed time constraints around certain solvers/native expansions, and bounded interval refinements. They are not interchangeable units. A user's `MaxTerms -> 1000` can mean a frontier size in one path, coefficient-tree size in another, and little protection against a dense optional cache or repeated unit powers in a third.[2, 8, 11, 12, 13]

Introduce an internal resource ledger with explicit counters, for example

```
<|"OutputBlocks" -> ..., "GeneratedProducts" -> ...,
  "Depth" -> ..., "CoefficientLeaves" -> ...,
  "DenseCoefficients" -> ..., "IntegerBits" -> ...,
  "ElapsedTime" -> ..., "Memory" -> ...|>
```

Budgets should be inherited by nested constructors, adapter calls, and refinement replays. Return structured exhaustion data naming the operation, consumed amount, and remaining certified precision. Do not retain a partially computed object with a stronger remainder claim than the completed work supports.

An inexpensive initial estimate can reject clearly excessive requests: for finitely generated positive weights, use $\prod_j (1 + \lfloor H/\delta_j \rfloor)$ as a conservative index-box bound, and account separately for

collisions that reduce output support. A box bound is not an exact work prediction, but it exposes tiny-gap inputs before substantial symbolic work is performed. A user override can permit large jobs without disabling all protection.

7.2 Eager factorization of logarithmic constants is another hot spot

The inspected `logCanon` rewrites logarithms of exact integers and rationals through `FactorInteger`. This can be helpful for identities such as $\log 12 = 2\log 2 + \log 3$, but it makes integer factorization part of ordinary coefficient normalization even when the result does not require it. Neither a small number of blocks nor a small `LeafCount` bounds the difficulty of factoring a large integer atom.[2]

This is a source-confirmed performance risk, not a timed failure claimed by this audit. Make prime-log canonicalization optional or bounded by integer bit length and an operation deadline. Preserve an unevaluated exact logarithm when the optimization budget is exhausted. Cache only within the fixed assumption/evaluation context, and distinguish "not reduced to the preferred canonical form" from "not mathematically admissible." Representation normalization should not force an unrelated expensive computation on every coefficient.

7.3 Where further optimizations are promising

Repeated merge-and-sort operations, repeated polynomial expansion, and repeated exact comparisons deserve profiles after the correctness fixes. Possible improvements include batched accumulation by exact weight, incremental sorted merges, a priority queue for the next boundary weight, memoized exact comparisons within one operation, and delayed coefficient simplification with normalization at stable boundaries. These are proposals, not demonstrated regressions in all current workloads.

The current incremental cache and cutoff-aware Fourier multiplication already address some obvious inefficiencies. In particular, the latest work should not be reviewed as though every Fourier product still materializes the full Cartesian product. An optimization proposal must first identify the current retained-product path and measure it against a matched-output baseline.[8, 10, 14]

7.4 How to read the repository's benchmark evidence

The 1.8.0 validation record reports the following parent/current measurements, using fresh 15.0.1 kernels, an immutable parent snapshot, one warmup, and three measured runs. These are *upstream-reported* results, not audit measurements.[14] The source record itself avoids a general

Workload	Parent seconds	Current seconds
100 coordinate inversions	0.12115	0.00418
Fourier merge fixture	0.19112	0.09396
200×200 Fourier product below weight 4	1.28634	0.00776
Public Fourier inverse fixture	0.02514	0.02325

Table 4: Selected upstream optimization evidence. The large microbenchmark gains do not imply corresponding end-to-end gains for every public inverse.

speedup claim. That restraint is appropriate: the public Fourier inverse example improves much less than the targeted product microbenchmark. The right next benchmark suite should report exact outputs or residuals alongside timing, not timing alone. It should cover cold and warm

loads, sparse rational and irrational supports, collisions, high log degrees, source-coordinate conversion, composite arithmetic, native special-function calls, and refinement from an already stored result.

7.5 Numerical stability is a separate concern

Exact symbolic output does not guarantee stable finite-precision evaluation. Direct `Normal[s]` can contain differences of large nearly equal terms, large exponential factors, or a Lambert-core ratio with a removable limiting form. Preserve logarithmic phase and scaling information for numerical evaluation; use `LogGamma` rather than forming large Gamma values when the equation has already been normalized logarithmically. A stable evaluator should expose its branch/domain check, chosen working precision, and an error estimate whose status is clear.

This is a development opportunity, not a claim that every current numerical checker is naive. The inspected certificate code already switches to a logarithmic phase for recognized transformed problems and uses exact rational targets/enclosures. Any new fast numerical route must remain distinct from that rigorous route.[13, 11]

8 Certificates, proof status, and numerical evidence

8.1 What the certificate machinery actually proves

The exact interval evaluator is one of the repository's strongest components. Its arithmetic uses rational endpoints and outward dyadic rounding. Exponentials are enclosed by range reduction, a positive Taylor sum, an explicit geometric majorant for the tail, and repeated interval squaring. Logarithms are reduced to a bounded interval and enclosed with the atanh series. Reciprocal and noninteger-power operations check their domains rather than applying unrestricted logarithmic identities. The certificate also checks the retained source condition on the entire closed verification interval. These are considerably stronger safeguards than simply evaluating a residual at high precision.[13]

The basic root argument is transparent. Let F be continuously differentiable on $[a, b]$, let $c \in (a, b)$, and suppose an interval calculation establishes a constant derivative sign and $|F'(x)| \geq \mu > 0$ throughout the interval. If $|F(c) - y| \leq \varepsilon$ and

$$[c - \varepsilon/\mu, c + \varepsilon/\mu] \subseteq [a, b],$$

then monotonicity, the mean value theorem, and the intermediate value theorem establish existence and uniqueness of a root in that bracket. Independently proved endpoint signs can establish existence when this symmetric bracket is too wide. Once existence is known, a signed derivative enclosure D sharpens the result through

$$x_* \in c - \frac{F(c) - y}{D}.$$

The implementation follows these alternatives and reports the kind of existence evidence used.[13]

8.2 Important limits that are already disclosed

The certificate does *not* turn every asymptotic remainder into a pointwise enclosure. It certifies the stored explicit forward expression on the verification interval, not every unknown function compatible with an `InputRemainder` declaration. Its returned fields explicitly state

`CertifiesInputRemainderFamily -> False`. A local unique root is not automatically a certificate of a globally chosen inverse branch. Moreover, the current elementary interval evaluator does not provide direct enclosures for arbitrary Gamma, Barnes, oscillatory, or special-function expressions. These are documented or source-visible scope limits, not evidence that the accepted elementary certificates are false.[13, 3]

The mathematical distinction should remain prominent in the interface. A formal inverse residual establishes consistency with an algebraic model; a numerical comparison supplies numerical evidence; an exact interval certificate establishes a pointwise theorem about an explicitly specified function and interval. None should silently upgrade the status of another. The current separation of `InverseResidual`, `InverseNumericalCheck`, and `InverseCertificate` is worth preserving.[2, 13]

8.3 F10: A required interval is presented with an Automatic default

Priority: Low; interface inconsistency.

The inspected `InverseCertificate` options include `"Interval" -> Automatic`, but the public body immediately requires an ordered pair of exact rational endpoints. Thus the default does not initiate interval discovery; it leads to `InvalidInterval`. Requiring an explicit interval is defensible and protects the proof boundary. The confusing part is an `Automatic` default that looks like an implemented selection algorithm.[13]

Prefer a clearly documented required verification interval and a diagnostic that says so, or add a separately bounded interval proposer. Such a proposer may use numerical heuristics, but only the independent exact checker should certify its output. Never equate a successful numerical bracket search with the final certificate.

8.4 Further certificate development

A useful next step is a certificate object whose proposition is explicit: equation, source interval, assumptions, target, arithmetic backend, and proof dependencies. Rational interval primitives could be isolated into a small independently tested module. Gamma and Barnes support would require admissible positive-domain enclosures and explicit bounds for the finite logarithmic asymptotic remainder; a symbolic reference to Stirling's expansion alone is insufficient. The DLMF states relevant asymptotic formulas and remainder conditions that can inform this work.[28, 29]

An even stronger long-term design would export a certificate transcript suitable for an independent checker, potentially in a proof assistant. This is a proposal, not a claim that the current symbolic simplifier or interval implementation has been formally verified. The practical first milestone is much smaller: audit-friendly rational arithmetic traces and property tests for every primitive.

9 API, user experience, and distribution

9.1 F08: The result-head rename needs a durable migration strategy

Priority: Medium; documented compatibility break.

Version 1.8.0 replaces `PowerLogSeries` with `GeneralizedSeries` and does not export the former head as an alias. The guide explicitly warns that saved input and explicit patterns must be updated. This is therefore a documented breaking change, not a hidden regression discovered by this audit. It nonetheless affects notebooks, stored expressions, downstream pattern matching, and user-written predicates.[1, 3]

A migration utility is preferable to blind textual replacement across notebooks. It should recognize a supported old schema, validate required fields, translate the head and changed properties, and attach a schema version. An optional temporary compatibility package can offer old-name patterns with deprecation messages without making two permanently independent public representations. Publish a compact migration example and tests showing how saved input from a previous release is handled.

9.2 One public head does not yet imply one complete contract

The guide correctly states that properties and operations vary by result family. Ordinary jets, factored expansions, reciprocal-logarithmic blocks, Gamma/Barnes inverse blocks, finite flat sectors, Fourier coefficients, and composite envelopes have different internal representations. A common wrapper makes them discoverable, but callers still need to know which properties are present and what `Cutoff` and `Terms` mean.[3, 5, 7]

Introduce a small required schema, such as `SchemaVersion`, `Variable`, `Approach`, `Assumptions`, `Domain`, `Expression`, `ErrorDescriptor`, `ScaleDescriptor`, `ProofStatus`, and `Capabilities`. Keep specialized fields under family-specific keys. A capability query should report whether composition, differentiation, integration, refinement, native conversion, or certification is available, and why a requested operation is unavailable. The existing property list can remain for detailed inspection.

The distinction between *not implemented*, *outside the admitted class*, *unproved domain*, *undecidable ordering*, and *resource exhaustion* should be reflected in structured failures. These categories have different remedies. For example, increasing `MaxTerms` cannot fix an unproved real branch; additional assumptions may fix a branch without changing the requested order.

9.3 Order specifications should carry their coordinate

The documented order conventions are sophisticated but easy to confuse. Ordinary cutoffs are exclusive exponents in a positive local coordinate. A factored result uses an order inside the correction bracket. A Gamma inverse uses powers of the retained inverse core. Marker depth and sector depth are inclusive counts, while structured special-function term goals count complete blocks separately for each carrier. The guide already explains these distinctions; the opportunity is to make them harder to misuse programmatically.[3]

A future typed request could distinguish `ExponentCutoff`, `BlockCount`, `MarkerDepth`, and `SectorDepth`, with an explicit scale identifier. Preserve the existing syntax as a convenience layer. Every result should retain the requested order, interpreted order, actual first omitted block, and effective precision separately. A cutoff beyond available precision must not be reported as newly achieved accuracy.

The finite-endpoint `"Power"` option also deserves a compact worked example. For $r \neq 1$, the ordinary interface describes a displacement observable, while $r = 1$ returns the translated original source variable. This is documented, but users can easily read `"Power" -> 2` as the square of

the full inverse. A dedicated observable expression, with an explicit source symbol and endpoint, would make that distinction visible.[3, 2]

9.4 Display and evaluation: preserve the successful choices

The read-only interpretation boxes are a sound decision: they show the finite expression and remainder while preserving the underlying object, and avoid allowing edited displayed coefficients to retain stale metadata. `Normal` deliberately discards the remainder. The guide makes both facts clear. Neither should be "fixed" by returning ordinary expressions everywhere.[2, 3]

The shorthand `s[value]` is documented as numerical substitution into the finite expression, not as a validated evaluation or a certified error bound. It could be supplemented by an explicit evaluator returning a value, domain-check outcome, precision information, and proof status. An optional display badge for a conditional, composite, or merely formal result would improve discoverability without cluttering every algebraic expression. The urgent prerequisite is F01: a badge is useful only when the assumptions it displays are complete.

9.5 F09: License identifiers disagree with the root license text

Priority: Low; distribution metadata.

The repository's root license says "MIT No Attribution License," whereas `PacletInfo.wl` declares `"License" -> "MIT"`. SPDX identifies the no-attribution variant as `MIT-0`. The maintainer should choose the intended license consistently and align the root text, source headers, paclet metadata, and release artifacts. This finding identifies inconsistent metadata; it does not determine the maintainer's intended licensing choice.[17, 16, 2, 30]

9.6 Paclet and release opportunities

The inspected paclet metadata declares a Kernel extension but no native documentation extension. The HTML/Markdown guide is substantial; adding installed reference pages, search integration, examples, and a versioned paclet artifact would make that work easier to reach from a notebook. A release should include a version, commit, standalone checksum, supported kernel/platform matrix, migration notes, and a validation manifest.[16, 3]

The current `Get[URLDownload[url]]` loading recommendation should remain. The repository records premature-EOF problems with direct HTTP loading of the large source and explains why downloading the complete file before `Get` is preferred. It also documents a withdrawn convenience-loader approach. Do not reintroduce that approach without reproducing and resolving its failure mode. For reproducible use, prefer a commit-pinned URL rather than mutable `main`.[15, 14]

10 Unimplemented capabilities and productive extensions

10.1 First build a scale protocol, not a universal promise

A realistic architecture is a family of scale implementations behind a shared protocol: normalize, compare weights, combine coefficients, transport errors, establish domains, refine from source evidence, and report available operations. The existing modules already contain much of this machinery, but dispatch and metadata vary. A protocol would reduce the chance that a new public route bypasses a check enforced elsewhere, as happened in F02. It would also make unsupported operations predictable instead of forcing callers to inspect internal `Kind` strings.[5, 6, 7]

Do not advertise closure under arbitrary transseries composition until the admitted support classes, coefficient algebras, and error contracts establish it. The current package is useful precisely because it handles selected real problems with explicit assumptions. Complex sectors, Stokes transitions, multivariate limits, uniform parameter limits, and unrestricted nested logarithmic/exponential scales should each have a stated scope and independent tests rather than being implied by the name `GeneralizedSeries`.

10.2 Integration would complement the existing calculus

The inspected public overview includes arithmetic, composition, differentiation, and refinement, but no general series-integration operation. A carefully scoped `SeriesIntegrate` is a natural extension. For an ordinary block and $\beta \neq -1$, an antiderivative is

$$\int w^{\beta} P(\log w)\, dw = w^{\beta+1} \sum_{j=0}^{\deg P} \frac{(-1)^j P^{(j)}(\log w)}{(\beta+1)^{j+1}}.$$

Differentiating the right-hand side makes adjacent terms telescope. For $\beta = -1$, the substitution $L = \log w$ reduces the integral to an ordinary polynomial antiderivative in L. These formulas are elementary and exact; the difficult API decision is the remainder and integration constant, not the finite blocks.[3]

For example, if $R(w) = \mathcal{O}(w^{\rho}(1+|\log w|)^k)$ and $\rho > -1$, then the endpoint-normalized remainder integral satisfies

$$\int_0^w R(t)\, dt = \mathcal{O}\big(w^{\rho+1}(1+|\log w|)^k\big).$$

To see this, substitute $t = ws$, bound $1 + |\log(ws)|$ by $(1+|\log w|)(1+|\log s|)$, and use the integrability of $s^{\rho}(1+|\log s|)^k$ on $(0,1)$. For $\rho \leq -1$, endpoint integration generally diverges; a primitive must be defined relative to a positive base point and carry the appropriate constant or divergent term. Coordinate Jacobians and exact prefactors also require explicit handling. Integration should therefore expose its normalization rather than pretending that all remainders integrate by incrementing an exponent.

10.3 Broaden operations only with the matching error algebra

Useful targets include addition and multiplication of compatible Gamma/Barnes inverse expansions in an ordered retained-core representation, rather than only through a conservative composite envelope; composition between selected reciprocal-logarithmic and ordinary scales; and refinement of operation trees with minimal recomputation. Each extension should define when two retained cores are equivalent and how errors are compared. Different exact carriers must not be merged merely because their finite samples look similar.[11, 5, 7]

For oscillatory coefficients, a lower bound on the magnitude of the leading coefficient is indispensable before inversion or division. A bounded sine or cosine amplitude can vanish infinitely often. Retaining absolute envelopes is therefore a feature, not a deficiency; sharper ordered arithmetic requires stronger hypotheses, such as a nonvanishing periodic coefficient with a certified lower bound.

10.4 Improve special-function coverage through explicit adapters

The native adapter is intentionally endpoint-specific and imports Wolfram's structured series rather than claiming an independent implementation of every special function. Continue adding narrowly specified adapters when they offer a clear improvement: explicit branch conditions,

a better retained core, certified tail bounds, or a supported operation that the generic import cannot provide. Fixed-parameter expansions should remain distinct from simultaneous parameter limits.[12, 3]

A useful capability catalogue would enumerate function family, source endpoint, parameter assumptions, available scale, native dependency, expansion nature, error status, and known unsupported cases. This is more informative than a long list of function heads. Automated examples should verify both successful cases and the expected refusal of singular parameter boundaries. Exact elementary identities should be simplified before invoking an asymptotic backend, but only under proved branch conditions.

11 Testing, continuous integration, and release confidence

11.1 F07: The current release record is focused, not full-suite validation

Priority: Medium; release assurance.

The 1.8.0 record states 305 passing tests in 18 selected files, 11 passing standalone-builder tests, and seven isolated standalone acceptance checks. It also states explicitly that the full package suite was not run. The preceding arithmetic refactor reports 316 passing tests in 17 selected files and likewise disclaims a full-suite run. These are valuable scoped records, not evidence that the entire repository is untested, but they are not a substitute for a full release gate after a public-head rename and shared arithmetic changes.[14, 1]

The inspected GitHub Actions workflow checks standalone freshness and runs Python builder tests on Ubuntu with Python 3.12. It does not run a Wolfram test suite. This audit makes that statement about the inspected workflow, not about unobserved private infrastructure. A licensed Wolfram runner, a protected scheduled workflow, or another controlled test service should provide the native gate. Untrusted pull-request code must not receive license credentials or other secrets.[18]

11.2 A regression suite organized around contracts

The archive includes eight desired-contract Wolfram tests. They are deliberately not described as an executed full MUnit suite. They test the assumption leak, equivalent observable routes, dense conversion protection, resource failure, polynomial and irrational inverse controls, and the exclusive cutoff convention. The successful native patch checks described in Section 12 are a smaller, explicitly recorded execution, not a substitute for all eight or for the upstream suite.

Several high-value metamorphic properties should be added to the maintained tests. Computing a result under an explicit condition and under the documented ambient-assumption mechanism should yield equivalent conditions and expressions. Adding a harmless outer constant to an observable must not change whether its real branch is admitted. Refinement should agree with a fresh construction to the same requested order; a failed refinement must leave the old state unchanged. Truncation should be idempotent at the same boundary and should never improve uncertainty without source evidence. Independent inverse methods should agree after complete equal-weight grouping. Polynomial reversion should agree with `InverseSeries` on an admitted analytic control family.[8, 21]

Cancellation tests must distinguish algebraic identity from independent uncertainty. Subtracting two unrelated approximations with equal finite parts does not cancel their unknown errors. On the other hand, reusing the same exact source expression may justify an identity if the implementation retains and proves that provenance. Test both cases and the held normalizer's

behavior around uncertain denominators. Derivative tests should include a small remainder with a large derivative, so that magnitude-only information cannot accidentally authorize differentiation.

11.3 Differential testing requires matched mathematical requests

A native comparison should match the approach, branch, retained scale, and actual output terms, not merely pass the same integer to two options named `SeriesTermGoal`. This audit's Erfc observation shows why. Compare finite expressions after converting to a common coordinate, and compare residual or error orders independently. A native result left unevaluated is a useful capability observation, but it does not prove that no substitution, transformation, or different native algorithm can solve the problem.[23, 24]

Property generators should favor small exact cases with known independent answers: polynomial inverses, exact monomial inverses, affine endpoint translations, matched power/logarithm observables, algebraic exponent collisions, and elementary interval enclosures. Add adversarial families for tiny positive gaps, large denominator LCMs, high logarithmic degree, contradictory parameter conditions, near-cancelled leading coefficients, pure remainders, and exact termination after several cancellations. These tests can be bounded without deliberately exhausting a kernel.

11.4 Validation records should be executable and immutable

The repository already records hashes and distinguishes selected suites, which is good practice. Extend each release manifest with the complete suite selection, kernel/platform, package loading route, source hashes before and after testing, messages, timeouts, skipped cases, and exact pass/fail counts. Separate transport failures from computational outcomes. A timeout should report the incomplete operation without converting it into a mathematical "unsupported" verdict.[14]

Measure cold loading separately from warm symbolic work. Keep output hashes, finite expressions, or independently checked residuals with each timing record. Use multiple runs and report dispersion. The one-run dense-cache measurements in this audit demonstrate a growth mechanism; they are not a portable performance ranking.

12 Included hardening candidate and reproducibility

12.1 What is supplied

The archive contains a hash-gated Python patch generator, bounded Wolfram reproducers, a native comparison script, desired-contract regression tests, a focused native patch-check script, Python component tests, and machine-readable observations. The generator accepts only the reviewed standalone distribution or the reviewed modular entry file. It never edits its input and refuses an unexpected source hash or mismatched replacement anchor. A successful invocation writes a candidate source, a unified diff, and a manifest to a new output directory.

The candidate makes four local changes: a shared uncertain-power guard, a bounded optional dense `SeriesData` cache, depth/support guards in the generic unit-series loops, and termination of the linear composition recurrence once its coefficient is identically zero. The dense-cache default is 100,000 coefficient positions. This is an explicit review policy, not a benchmark-derived universal optimum or a replacement for a future operation-wide memory budget.

12.2 What was actually checked

The equivalent replacements were applied to the pinned standalone source inside the native Wolfram evaluator. Replacement counts were $1, 2, 1, 1$ for the pure-remainder, dense-allocation, generic-series, and composition anchors respectively. The patched source was loaded and the following five focused checks succeeded:

Check	Observed patched result
Wrapped uncertain square root	`Failure["UnknownLeadingTerm", ...]`
Tiny-gap unit-series budget	`Failure["ResourceLimit", ...]`
Million-position optional dense cache	`Missing["DenseRepresentationLimit", ...]`; the two-term sparse expression is preserved
Polynomial inverse control	$y - y^2 + 2y^3 - 5y^4 + 14y^5$, unchanged
Annihilated composition recurrence	Constant jet $\{\{0, 1\}\}$

The native candidate's SHA-256 is recorded in `evidence/native_patch_verified.json`. Eighteen Python tests also passed: they check exact rational resource calculations and the patch generator's mechanics on explicit source-anchor fixtures. They do *not* execute Wolfram Language or constitute a full-source semantic validation.

12.3 What remains unvalidated or unfixed

The full upstream suite was not run against either the original or patched source in this audit. The ambient-assumption defect F01 is *not fixed* by this candidate. Its proper repair spans entry evaluation, arithmetic, stored metadata, and refinement replay, and should not be disguised as a one-line source substitution. Global resource accounting, the expensive logarithm-normalization policy, every scale-specific path, and cross-platform behavior also remain outside the local patch's validation scope.

The patch is therefore a review candidate, not a certified release. It changes failure behavior intentionally: a request that previously exceeded a nominal resource budget may now fail early, and a wrapped fractional observable with no sign evidence is rejected. The maintainer should run the full native suite and review those changed failures before merging. Changes to the canonical modular entry must be followed by regeneration of the standalone source; editing only the generated distribution would be lost on the next build.[15]

12.4 Running the artifacts

From the extracted archive, the Python component tests require only the standard library:

```
python -m unittest discover -s code -p test_audit.py -v
```

A local Wolfram installation can run the bounded observations and comparisons:

```
wolframscript -file code/reproduce.wl
wolframscript -file code/comparison.wl
wolframscript -file code/native_patch_check.wl
```

These scripts retrieve and verify the pinned source before loading it. To generate review files from a separately downloaded original:

```
python code/harden_source.py /path/to/AsymptoticInverse.wl \
  --out-dir audit_patch
wolframscript -file code/reproduce.wl \
  audit_patch/AsymptoticInverse.audit.wl --allow-modified
```

Read the archive's README for the distinction between the original-source reproduction script and the desired-contract regression tests. Re-running a script produces new observations on the local kernel; it does not overwrite the historical evidence shipped with this audit. The full upstream repository and its original test suite are not bundled.

13 Prioritized development roadmap

13.1 Milestone A: restore complete semantic context

Make the assumption context explicit, immutable, and shared by all constructors and operations. Fix F01 and F02, add equivalent-route tests, and distinguish real-germ success from a complex-valued magnitude estimate. Acceptance requires the ambient-assumption regression to preserve $a > 0$ or refuse the request, the wrapped square-root test to match the direct power policy, and refinement under a changed ambient context not to rewrite the original proposition silently.

13.2 Milestone B: prevent sparse requests from becoming unbounded work

Fix eager dense conversion, adopt operation-wide budgets, and terminate annihilated coefficient recurrences. Make cache construction optional and lazy. Acceptance requires a two-term input with a huge denominator to return a sparse object or a controlled resource failure without allocating the full grid, and tiny-gap expansion requests to obey a documented work budget. Changing an optional cache limit must not change the mathematical finite expression or remainder.

13.3 Milestone C: make release evidence comprehensive

Run the full native suite after the head rename and hardening changes, retain focused fast gates, and add a supported-version/platform matrix. Acceptance requires reproducible manifests, source immutability checks, standalone freshness, explicit skip/timeout accounting, and a release artifact linked to its tested commit. The existing focused records are a foundation to extend, not material to discard.

13.4 Milestone D: stabilize the public object and migration path

Version the result schema, publish the capability matrix, and add migration tooling. Separate order requests from achieved precision and distinguish required domains from optional presentation metadata. Acceptance requires old supported saved objects to migrate predictably, unsupported operations to fail with actionable categories, and native conversion to advertise any lost logarithmic or branch information.

13.5 Milestone E: optimize measured workloads and complete the calculus

Profile exact merges, coefficient normalization, refinement replay, and native adapter calls using matched outputs. Add scoped integration and additional ordered operations only with proved

error transport. Acceptance requires an independent correctness comparison for each optimization and explicit remainder contracts for every new operation; a smaller timing alone is not sufficient.

13.6 Milestone F: extend certified and advanced scales deliberately

Add special-function interval enclosures, independent certificate checking, selected complex sectors, and uniform parameter limits only as separately specified projects. Prioritize concrete examples where the current result objects and retained cores provide an advantage. Acceptance should be expressed as a finite, documented domain of supported problems with proofs or certified bounds, not as a broad claim of universal transseries support.

14 Conclusion

The repository provides genuine value beyond a thin wrapper around `Series`. Its exact real-exponent inverse calculus, complete-block enumeration, retained Lambert cores, explicit error envelopes, and elementary interval certificates deserve continued development. The package also relies productively on native Wolfram functionality, especially for special-function series; comparison should acknowledge that dependency rather than frame the systems as mutually exclusive alternatives.

The audit's most important result is actionable: four issues were reproduced in a native 15.0.0 kernel, and a limited hardening candidate passed five focused native checks without changing the polynomial inverse control. The most serious remaining repair is complete assumption provenance. Stabilizing that contract, enforcing resource limits throughout the computation, and running a full release suite will do more for trustworthiness than adding another large function-name list. Once those foundations are secure, a common scale protocol, better migration and capability discovery, integration, and independently checkable certificates provide a coherent path forward.

A Source map and evidence interpretation

The references below are pinned to the reviewed commit. This is a map of the principal inspected source areas, not a claim that every repository line or archived research report was exhaustively reviewed.

Area	Principal inspected material and purpose
Public entry and exact jets	`Kernel/AsymptoticInverse.wl`: coefficient normalization, sparse jets, generic series loops, forward/inverse construction, eager native conversion, accessors, display, residuals, and module loading.
Ordered arithmetic	`SeriesOperations.wl` and `SeriesArithmetic.wl`: representation conversion, powers/observables, composition, derivative contracts, held normalization, and fallback dispatch.
Composite arithmetic	`SeriesEnvelopeArithmetic.wl`: approach recovery, nonnegative absolute errors, and resource checks.

Area	Principal inspected material and purpose
Inverse state	`IncrementalInverse.wl` and `ExactTermination.wl`: weighted frontiers, caching, Newton checks, grouped coefficients, and original-source termination verification.
Additional coefficients/cores	`FourierCoefficients.wl` and `GammaInverse.wl`: frequency grouping, retained-pair products, inverse core construction, and family-specific metadata.
Native special functions	`NativeSpecialFunctions.wl`: structured native import, real projection, absolute majorants, per-carrier truncation, and exact-equality checks.
Certificates	`InverseCertificates.wl`: rational interval primitives, source-domain checks, root proof, adaptive control, and scope statements.
Documentation and release	README, UserGuide, mathematical article entry/abstract, development notes, current validation record, paclet metadata, root license, and standalone CI workflow.

The evidence JSON files record successful native observations and their kernel/source identity. Tool transport errors are disclosed as limitations and are not counted as failed mathematical tests. The Python test log records only the tests actually executed in the container. `findings.csv` is a compact index of the findings, not a substitute for their hypotheses and qualifications in the article. The source patch generator includes short upstream source fragments solely to anchor and review the proposed edits.

B Reference sources

All repository references below identify commit `07a9781212beb2eeb9ff16aa625b50ac27974078`. Public documentation was consulted on 9 September 2026. Native observations are supplied in the accompanying evidence files and should be interpreted on the recorded kernel, not as claims about every Wolfram release.

References

[1] V. Reshetnikov, *Asymptotic*, commit 07a9781, "Rename the result head to GeneralizedSeries and simplify series operations," 9 September 2026. Pinned commit and change record.

[2] *AsymptoticInverse kernel entry and exact power–log engine*. Pinned source.

[3] *AsymptoticInverse User Guide*, version 1.8.0. Pinned Markdown guide.

[4] V. Reshetnikov, *Real asymptotic expansions and inverse functions: Power–logarithmic series, real branches, and error transport*. Pinned article source entry and abstract.

[5] *SeriesOperations.wl*. Pinned source.

[6] *SeriesArithmetic.wl*. Pinned source.

[7] *SeriesEnvelopeArithmetic.wl*. Pinned source.

[8] *IncrementalInverse.wl*. Pinned source.

[9] *ExactTermination.wl*. Pinned source.

[10] *FourierCoefficients.wl*. Pinned source.

[11] *GammaInverse.wl*, including shared Gamma/Barnes construction. Pinned source.

[12] *NativeSpecialFunctions.wl*. Pinned source.

[13] *InverseCertificates.wl*. Pinned source.

[14] *Review and validation record*. Pinned validation README.

[15] *Development and provenance*. Pinned development notes.

[16] *PacletInfo.wl*. Pinned metadata.

[17] *MIT No Attribution License*, repository root LICENSE. Pinned license text.

[18] *Standalone package freshness workflow*. Pinned GitHub Actions workflow.

[19] Wolfram Research, *Series*. https://reference.wolfram.com/language/ref/Series.html.

[20] Wolfram Research, *SeriesData*. https://reference.wolfram.com/language/ref/SeriesData.html.

[21] Wolfram Research, *InverseSeries*. https://reference.wolfram.com/language/ref/InverseSeries.html.

[22] Wolfram Research, *ComposeSeries*. https://reference.wolfram.com/language/ref/ComposeSeries.html.

[23] Wolfram Research, *Asymptotic*. https://reference.wolfram.com/language/ref/Asymptotic.html.

[24] Wolfram Research, *AsymptoticSolve*. https://reference.wolfram.com/language/ref/AsymptoticSolve.html.

[25] Wolfram Research, *Simplify*, assumptions and options. https://reference.wolfram.com/language/ref/Simplify.html.

[26] Wolfram Research, *Assuming*. https://reference.wolfram.com/language/ref/Assuming.html.

[27] NIST, *Digital Library of Mathematical Functions*, Section 2.1, definitions and properties of asymptotic approximations. https://dlmf.nist.gov/2.1.

[28] NIST DLMF, Section 5.11, Gamma-function asymptotic expansions and error bounds. https://dlmf.nist.gov/5.11.

[29] NIST DLMF, Section 5.17, Barnes' G-function. https://dlmf.nist.gov/5.17.

[30] SPDX, *MIT No Attribution*, identifier MIT-0. https://spdx.org/licenses/MIT-0.html.